Sales and Customer Analytics System

Introduction

Vilvah is a DBMS project based on a natural skincare and personal care e-commerce business. It is designed to manage customer details, products, suppliers, inventory, orders, payments, reviews, and ratings. The database helps organize information, reduce data redundancy, improve accuracy, and support efficient business operations.

Objectives

1. Apply COUNT(), SUM(), AVG(), MIN(), and MAX().
2. Generate total sales reports.
3. Find top customers based on purchase amount.
4. Identify best-selling products.
5. Perform category-wise sales analysis.

1. Aggregate Functions

Definition:

Aggregate functions are SQL functions used to perform calculations on a group of rows and return a single result.

1.1 COUNT()

Find the total number of orders:

```
SQL> SELECT COUNT(Order_ID) AS Total_Orders
  2  FROM Orders;

TOTAL_ORDERS
------------
          15
```

1.2 SUM()

Find the total sales amount:

```
SQL> SELECT SUM(Total_Amount) AS Total_Sales
  2  FROM Orders;

TOTAL_SALES
-----------
      11341
```

1.3 AVG()

Find the average order amount:

```
SQL> SELECT AVG(Total_Amount) AS Average_Order_Amount
  2  FROM Orders;

AVERAGE_ORDER_AMOUNT
--------------------
          756.066667
```

1.4 MIN()

Find the lowest order amount:

```
SQL> SELECT MIN(Total_Amount) AS Minimum_Order
  2  FROM Orders;

MINIMUM_ORDER
-------------
          399
```

1.5 MAX()

Find the highest order amount:

```
SQL> SELECT MAX(Total_Amount) AS Maximum_Order
  2  FROM Orders;

MAXIMUM_ORDER
-------------
          1200
```

2. Generate Total Sales Reports

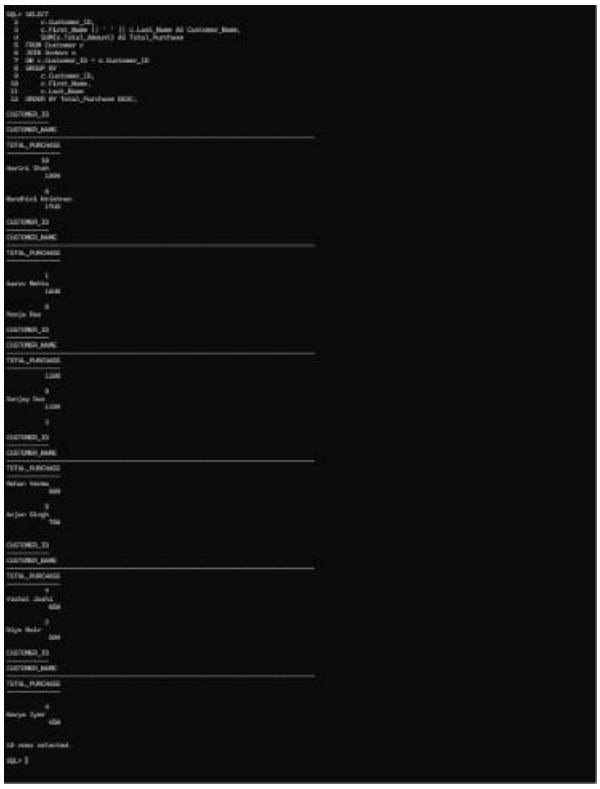
A sales report can be generated using aggregate functions on the Total_Amount column.

```
SQL> SELECT
  2      COUNT(Order_ID) AS Total_Orders,
  3      SUM(Total_Amount) AS Total_Sales,
  4      AVG(Total_Amount) AS Average_Order_Value,
  5      MIN(Total_Amount) AS Minimum_Order,
  6      MAX(Total_Amount) AS Maximum_Order
  7  FROM Orders;

TOTAL_ORDERS TOTAL_SALES AVERAGE_ORDER_VALUE MINIMUM_ORDER MAXIMUM_ORDER
------------ ----------- -------------------- ------------- -------------
          15       11341          756.066667           399          1200
```

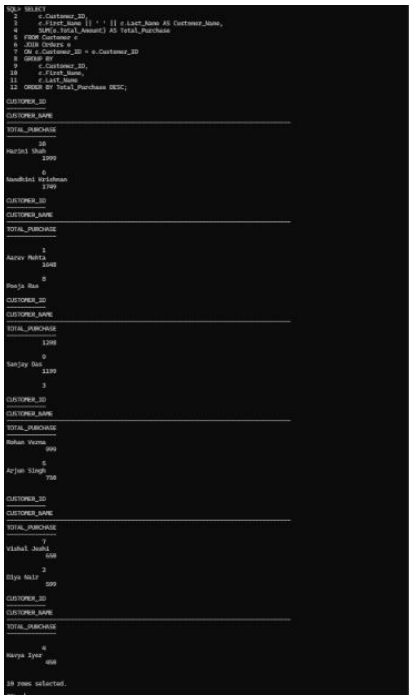
2.1 Customer-wise Sales Report

To find how much each customer has purchased:



3. Find Top Customers Based on Purchase Amount

The SUM() function can be combined with GROUP BY and ORDER BY to identify customers with higher purchase amounts.



4. Identify Best-Selling Products

The Order_Details table contains the quantity of each product ordered.

Therefore, we can use SUM(Quantity) to find the total quantity sold for each product.

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      SUM(od.Quantity) AS Total_Quantity_Sold
  5  FROM Product p
  6  JOIN Order_Details od
  7  ON p.Product_ID = od.Product_ID
  8  GROUP BY
  9      p.Product_ID,
 10      p.Product_Name
 11  ORDER BY Total_Quantity_Sold DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
      1009
Aloe Vera Body Lotion
                   2
```

4.1 Product Sales Amount

Instead of only calculating quantity, we can calculate the total sales generated by each product.

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      SUM(od.Quantity) AS Total_Quantity_Sold,
  5      SUM(od.Total_Price) AS Total_Sales
  6  FROM Product p
  7  JOIN Order_Details od
  8  ON p.Product_ID = od.Product_ID
  9  GROUP BY
 10      p.Product_ID,
 11      p.Product_Name
 12  ORDER BY Total_Sales DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------
      1009
Aloe Vera Body Lotion
                   2         698
```

5. Category-wise Sales Analysis

Category-wise analysis helps determine the sales generated by each product category.

```
SQL> SELECT
  2      c.Category_ID,
  3      c.Category_Name,
  4      SUM(od.Quantity) AS Total_Quantity_Sold,
  5      SUM(od.Total_Price) AS Total_Sales
  6  FROM Category c
  7  JOIN Product p
  8  ON c.Category_ID = p.Category_ID
  9  JOIN Order_Details od
 10  ON p.Product_ID = od.Product_ID
 11  GROUP BY
 12      c.Category_ID,
 13      c.Category_Name
 14  ORDER BY Total_Sales DESC;

CATEGORY_ID CATEGORY_NAME
----------- --------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_SALES
------------------- -----------
          2 Body Care
                  2         698
```

6. Combined Business Sales Report

A complete sales report can combine customer, order, product, and category information.

```
SQL> SELECT
  2      c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3      o.Order_ID,
  4      p.Product_Name,
  5      cat.Category_Name,
  6      od.Quantity,
  7      od.Total_Price
  8  FROM Customer c
  9  JOIN Orders o
 10  ON c.Customer_ID = o.Customer_ID
 11  JOIN Order_Details od
 12  ON o.Order_ID = od.Order_ID
 13  JOIN Product p
 14  ON od.Product_ID = p.Product_ID
 15  JOIN Category cat
 16  ON p.Category_ID = cat.Category_ID
 17  ORDER BY o.Order_ID;

CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGORY_NAME                                       QUANTITY TOTAL_PRICE
--------------------------------------------------- ---------- -----------
Aarav Mehta
       101
Aloe Vera Body Lotion
Body Care                                                    2         698
```